Quick Start Guide

WDissector fuzzer is the new iteration of the older GreyHound fuzzer. It now offers fuzzing integration with wireshark to extend supported protocols and performance. This quick start guides on how to download and run the binary releases and alternativelly how to compile it from sources. It includes by default a custom version of Wireshark with some new dissectors.

> **Repository Link**
> WDissector repository is maintained on GitLab and requires authorisation. You can access the repository here: **https://gitlab.com/asset-sutd/software/wireless-deep-fuzzer/-/tree/wdissector**
>
> Note that WDissector is on `wdissector` branch.

Compatible OS

> **OS Compatibility**
> The release has been tested and confirmed to work on both **Ubuntu 18.04** and **Ubuntu 20.04**.

1) Download Binary Release

The latest release can be downloaded either via Web Browser or via command line.

- Via **Web Browser** (Requires GitLab login): **https://gitlab.com/asset-sutd/software/wireless-deep-fuzzer/-/jobs/artifacts/wdissector/raw/release/wdissector.tar.zst?job=release**
- Or Via **bash command line**:

```sh
wget --header="PRIVATE-TOKEN: -5AEDnLgc6u-SV_zPAY5" \
--content-disposition "https://gitlab.com/api/v4/projects/\
asset-sutd%2Fsoftware%2Fwireless-deep-fuzzer/jobs/artifacts/\
wdissector/raw/release/wdissector.tar.zst?job=release"
```

Next, extract the downloaded release file (wdissector.tar.zst) as follows:

```sh
# Install zstandard
sudo apt install zstd
```

```
# Extract the wdissector compressed file
tar -I zstd -xf wdissector.tar.zst
cd wdissector
```

1.1) Compile from source (optional)

Several requirements needs to be installed before compiling the project. An automated script for Ubuntu 18.04/20.04 is provided on `requirements.sh` . To compile from source, simply run the following commands:

```sh
git clone https://gitlab.com/asset-sutd/software/wireless-deep-fuzzer -b wdissector --d
cd wdissector
./requirements.sh dev # Install all requirements to compile wdissector from source
./requirements.sh doc  # Install nodejs requirements to generate documentation
./build.sh all # Compile all binaries. It may take around 15min. Go get a coffe!
./build.sh doc # Generate documentation
```

2) Flash BT firmware

This step is required only for BT fuzzing. The custom BT firmware needs to be uploaded to an ESP32 board with FT232H or FT2232H high speed serial to usb chip. The recommended board to use is **ESP-WROVER-KIT-VE**

Download the ESP32 firmware package as follows:

- Via **Web Browser** (Requires GitLab login): **https://gitlab.com/asset-sutd/software/wireless-deep-fuzzer/-/jobs/artifacts/wdissector/raw/release/esp32driver.zip?job=release**
- Or Via **bash command line**:

```sh
wget --header="PRIVATE-TOKEN: -5AEDnLgc6u-SV_zPAY5" \
--content-disposition "https://gitlab.com/api/v4/projects/\
asset-sutd%2Fsoftware%2Fwireless-deep-fuzzer/jobs/artifacts/\
wdissector/raw/release/esp32driver.zip?job=release"
```

You can upload the firmware to ESP32 on either Windows or Linux. For windows you need to manually install python3 before executing the installation script.

Windows Linux (Ubuntu)

```sh
sudo apt install unzip python3-dev
unzip esp32bt.zip # Extract esp32bt.zip if you haven't done so
cd release
python3 firmware.py /dev/ttyUSB0 # Please change your serial port to match your device.
```

> **WARNING**
> You may need to press and hold the `boot` button during the firmware upload process if you get failures.

3) Running the fuzzer

Before running the fuzzer, you need to install few system libraries that the binary release requires. You can do so by executing `requirements.sh` on wdissector root folder. The steps are shown below:

```sh
# Install few requirements
./requirements.sh
# Run one of the following binaries
sudo bin/bt_fuzzer # BT Fuzzer
sudo bin/bt_fuzzer --no-gui # BT Fuzzer without GUI interface
sudo bin/lte_fuzzer # LTE Fuzzer
bin/wireshark # Standalone Wireshark GUI with custom dissectors (Use this to see the lo
```

4) Usage

- **Fuzzers -** See more options and usage on **Fuzzers** section.
- ☁ **Web API -** Integrate fuzzing to your workflow via the integrated SocketIO server. Examples on how to control the fuzzer or receive events remotely is presented on section **Web API**.

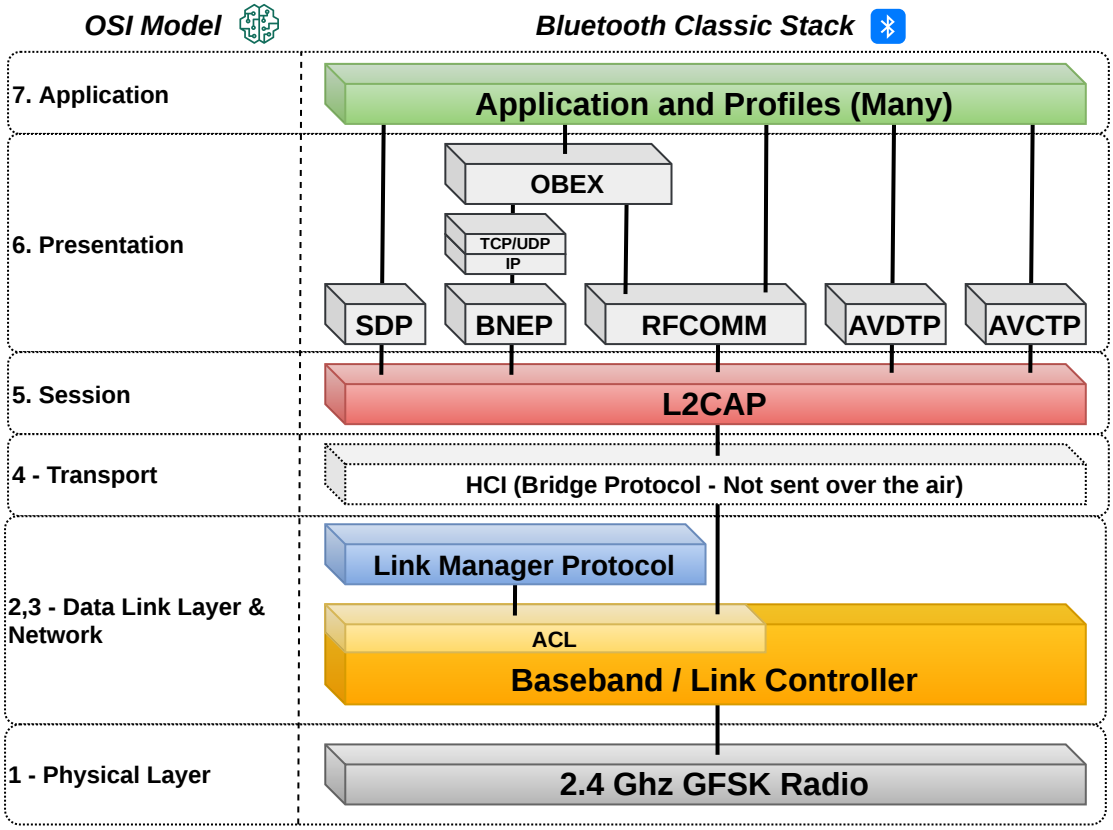
✉ Protocol Stacks

WDissector is built upon well known protocols stack implementation. These are used to generate messages and to guide the target device towards a set of protocol procedures which are expected to be tested again unknown or insecure behaviour.

Bluetooth Classic

Implementation:

- Layer 1-2:
  - Baseband & LMP - **ESP32 Bluetooth Library** (Reverse Engineered)
- Layer 2-7:
  - L2CAP and profiles - **BlueKitchen** (Open Source)



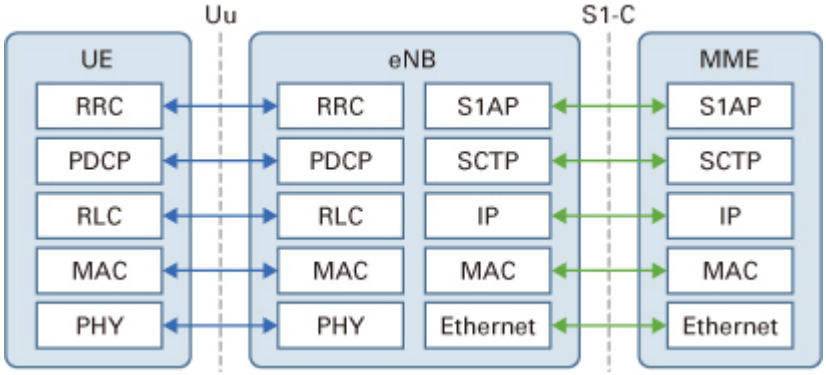
4G / LTE

Implementation:

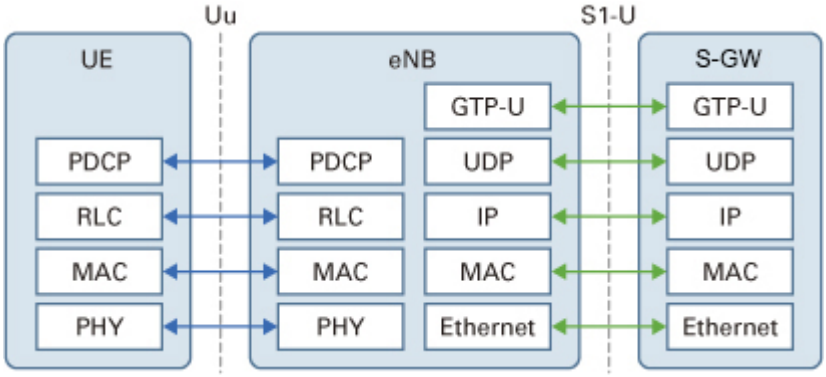
- Layer 1-2:

- eNB (Base-station) - **Open Air Interface** (Open Source)
- Layer 3 -7:
  - Core Network - **Open5GS** (Open Source)

Control Plane



User Plane



Fuzzers

Bluetooth Classic

You can start the fuzzer as follows:

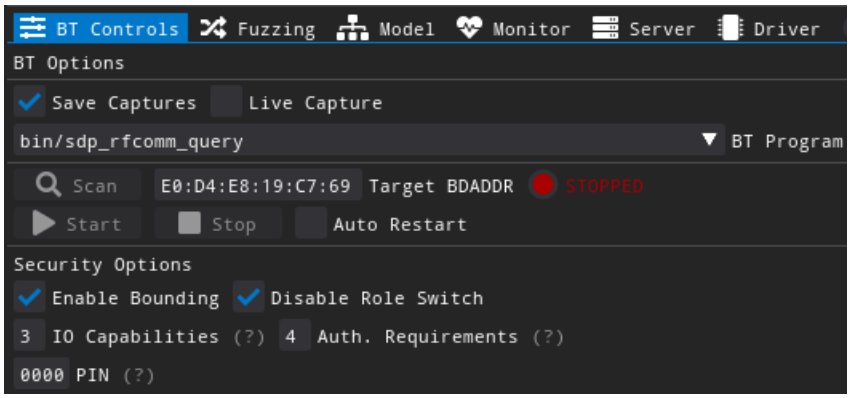
```sh
sudo bin/bt_fuzzer --scan # Scan for targets (BDAddress) for 15 seconds
sudo bin/bt_fuzzer # Start fuzzer with graphical user interface (GUI)
sudo bin/bt_fuzzer --no-gui --autostart --target=E8:D0:3C:94:2C:66  # Start fuzzer with
```

BT Command line options

```sh
sudo bin/bt_fuzzer --help
Bluetooth Classic Fuzzer (Baseband, LMP, L2CAP, etc)
Usage:
  BT Fuzzer [OPTION...]

      --help                Print help
      --default-config      Start with default config
      --autostart           Automatically start (default: true)
      --no-gui              Start without GUI
      --test-webview        Test GUI webview performance (requires internet)
      --live-capture        Open wireshark in live capture mode
      --exploit [=arg(=)]   Exploit Name
      --list-exploits       List all exploits
      --host arg            Host BDAddress
      --host-port arg       Host serial port name of BT Interface
                            (ESP-WROVER-KIT)
      --random_bdaddress    Enable/Disable host BDAddress randomization
      --target arg          Target BDAddress (default: /dev/ttyUSB1)
      --target-port arg     Target serial port name to detect crashes
                            (default: /dev/ttyUSB2)
      --target-baud arg     Target baud rate (default: 115200)
      --bounding            Enable/Disable Bounding (default: true)
      --iocap arg           IO Capabilities (default: 3)
      --authreq arg         Authentication Request flag (default: 3)
      --scan                Scan BT Targets
```

BT Options



- **Save Captures** (Checkbox) - When option is enabled, capture file is saved on `logs/Bluetooth/capture_bluetooth.pcapng`
- **Live Capture** (Checkbox) - Opens Wireshark in live capture mode
- **Scan** (Button) - Scan for BT targets **(TODO)**
- **BT Program** (Combo box) - Programs or "Profiles" which connect with a target device or wait for a connection. The available BT programs are available on the table below:

BT Program	Connection Type	PROFILE
bin/sdp_rfcomm_query	Initiator / Master	SDP / RFCOMM
bin/spp_counter	Advertiser / Slave	SDP / RFCOMM
bin/a2dp_sink_demo	Advertiser / Sound Sink	A2DP
bin/a2dp_source_demo	Initiator / Sound Source	A2DP

- **Target BDADDR** (Text box) - Target of the device to connect. Only applied for BT programs which **initiates** the connection. This is not applied for programs that wait for connections such as `bin/spp_counter` or `bin/a2dp_sink_demo` .
- **Start / Stop** (Buttons) - Start or stop the previously selected BT program
- **Auto Restart** (Checkbox) - Automatically restart BT program in case of a hang **(WIP)**

Security Options

- **Enable Bounding** (Checkbox) - Enabled BT Pairing. If disable all the next options has no effect.

- **Disable Role Switch** (Checkbox) - Forces connection to reject any attempts to perform role switching. This ensure that once the master connects to a slave, their roles stay the same during the session. Exploits such as KNOB requires this so the master (being the fuzzer) can mutate the `LMP_max_encryption_key_size_req` packet. **Disable this options if the slave does not accept the connection without role switching.**

- **IO Capabilities** (Text box) - Selects IO capabilities of the fuzzer during the pairing process according to the following:

  - Display Only = 0
  - Display Yes No = 1
  - Keyboard Only = 2
  - No Input No Output = 3 (Default)
  - Unknown = 256
- **Auth. Requirements** (Text box) - Flag which indicates the authentication parameters during the pairing process.

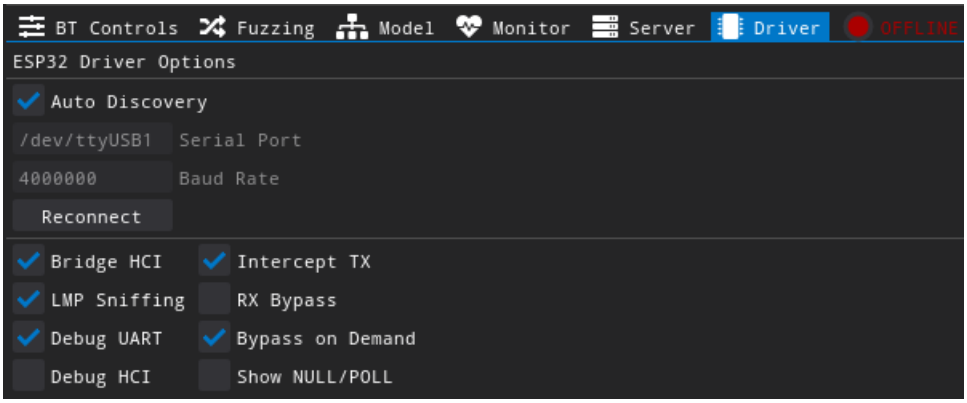
  - No MitM, No Bouding = 0
  - MitM, No Bouding = 1
  - No MitM, Dedicated Bouding = 2
  - MitM, Dedicated Bouding = 3
  - No MitM, General Bouding = 4
  - MitM, General Bouding = 5
- **PIN** (Text box) - 4 digit PIN number to be used during pairing (legacy pairing method).

The aforementioned BT options are loaded from `configs/bt_config.json` on the following attributes:

```json
{
    "config": {
        "Bluetooth": {
            // BT Options
            "EnableBounding": true,
            "AuthReq": 4,
            "DisableRoleSwitch": true,
            "IOCap": 3,
            "Pin": "0000",
            "TargetBDAddress": "E0:D4:E8:19:C7:69",
            // TODO: Store a list of targets
            "TargetBDAddressList": [
                "24:0A:C4:61:1C:1A",
                "E0:D4:E8:19:C7:69"
            ]
            // ...
    }
```

```
  // ...
}
```

Driver Options



- **Auto Discovery** (Checkbox) - Tries to auto-discover which serial port the ESP32 is connected by probing a version string. Disable this if facing any issues with ESP32 being recognised.
- **Serial Port** (Text box) - Defines serial port path for ESP32. This is automatically changed when **Auto Discovery** is enabled.
- **Baud Rate** (Text box) - Defines baud rate of the chosen serial port for ESP32. It defaults to **4000000** and is automatically changed when **Auto Discovery** is enabled.
- **Reconnect** (Button) - Attempts re-connection with serial port. This is usually not required as the fuzzer tries to re-open the serial port each second.

Advanced Driver Options

> **WARNING**
> The defaults for advanced options are recommended to be used unless debugging the firmware or manually experimenting with BT.

- **Bridge HCI** (Checkbox) - Creates raw pseudo-terminal (serial bridge) which can be used to connect a BT stack to the fuzzer. The pseudo terminal is created on /dev/pts folder and advertised by the fuzzer on the "**Events**" window.

- **LMP Sniffing** (Checkbox) - Allows firmware to send LMP packets. This option is required for the fuzzer. It can only be disabled if using ESP32 as a standalone HCI BT device. Not only LMP packets are sniffed but also relevant baseband information such as BT header, channel number, device role, channel speed (table type - ptt) and TX/RX encryption.

> **Info**
> ESP32 is a common HCI device at startup, but the fuzzer enables LMP Sniffing mode for its use. This means that you can integrate this ESP32 firmware into standard HCI tests by

> enabling `Bridge HCI` and `LMP Sniffing` then sending raw HCI commands on the pseudo-terminal created by the fuzzer (on `/dev/pts/`).

- **Debug UART** (Checkbox) - Outputs serial ASCII messages coming from the firmware. This is useful to get crash traces from the firmware.
- **Debug HCI** (Checkbox) - Prints HCI hex encoded messages between host and firmware.
- **Intercept TX** (Checkbox) - Intercept BT packets from the firmware before they are sent over the air. This is required for "**Packet Mutation**" to work.

> **Low Latency Required**
> Intercept TX requires a high speed USB adaptor such as FT2232H. As a general rule, a good latency is below 600us.

- **RX Bypass** (Checkbox) - Completely disables the firmware to respond to LMP packets while a connection is maintained with the target BT device. This can be used to send arbitrary LMP packets from the host since the very first LMP interactions without interference from ESP32 own BT stack.

> **TIP**
> RX Bypass is a **powerful feature** which can be used for many used besides fuzzing. Applications range from exploits, protocol compliance testing, BT experimentation, LMP debugging over-the-air, etc. This feature is quite stable.

- **Bypass on demand** (Checkbox) - Temporarily disable LMP reception on the firmware when duplicated packets are sent. This prevents random firmware crashes as ESP32 own BT stack may not be prepared to receive out-of-order response packets from the target device.
- **Show NULL / POLL** (Checkbox) - (**Experimental feature**) Enables receiving BT baseband POLL and NULL messages between ESP32 and target device. This packets are also saved to BT capture file and are shown on the "TX / RX" view window. Depending on the Poll time, it can log too many packets, thus significantly increasing the BT capture file.

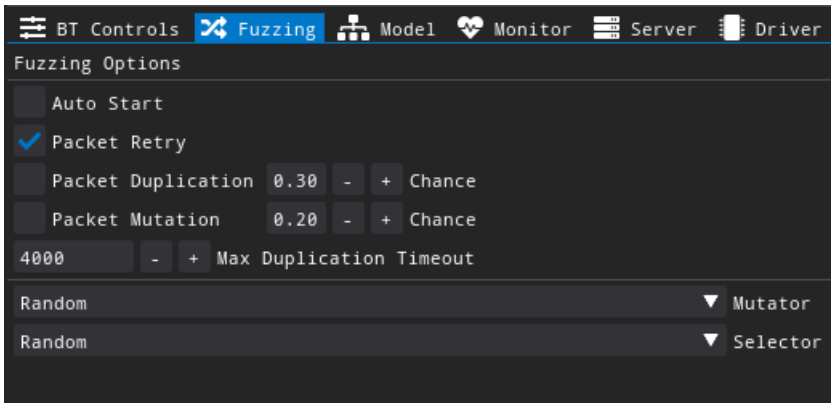
The aforementioned advanced BT driver options are loaded from `configs/bt_config.json` on the following attributes:

```json
{
    "config": {
        "Bluetooth": {
            "BridgeHCI": true,
            "InterceptTX": true,
            "LMPSniffing": true,
            "RXBypass": false,
            "RXBypassOnDemand": true,
```

```
            "SaveHCIPackets": false,        // Saves the HCI only capture from BT stack
            "SerialAutoDiscovery": true,
            "SerialBaudRate": 4000000,
            "SerialEnableDebug": true,
            "SerialEnableDebugHCI": false,
            "SerialPort": "/dev/ttyUSB1",
            "ShowNullPollPackets": false
            // ...
        },
      // ...
    }
}
```

Common Options

Fuzzing

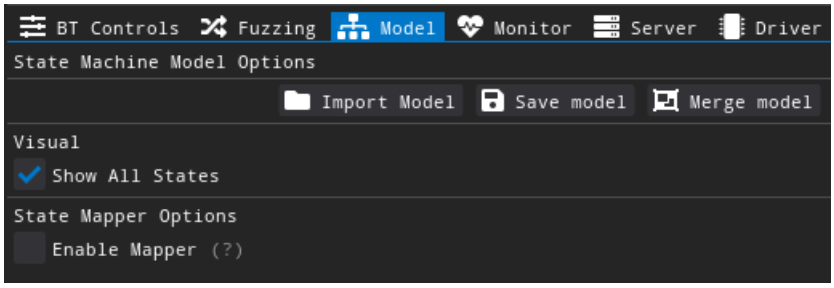


- **Auto Start** (Checkbox) - Automatically starts fuzzing a selected target (via GUI or config. file) when starting the fuzzer
- **Packet Retry** (Checkbox) - Resend packets that are not replied by the target. Timeout in milliseconds can be adjusted via parameter `PacketRetryTimeoutMS` on `configs/bt_config.json` .
- **Packet Duplication** (Checkbox & Number) - Enables packet duplication (out of order). The "Chance" number defines the probability per packet for a duplication to occur in the future.
- **Packet Mutation** (Checkbox & Number) - Enables packet mutation. The "Chance" number defines the probability per packet for a mutation to occur.
- **Max Duplication Timeout** (Text box & Number) - This defines the maximum time for a duplicated to be sent after the original.
- **Mutator** (Combo box) - Defines mutation operator **(WIP)**
- **Selector** (Combo box) - Defines selector for the operators **(WIP)**

The aforementioned fuzzing options are loaded from `configs/bt_config.json` on the following attributes:

```json
{
    "config": {
        // ...
        "Fuzzing": {
            "DefaultDuplicationProbability": 0.3,
            "DefaultMutationProbability": 0.2,
            "MaxDuplicationTime": 4000,
            "PacketRetry": true,
            "PacketRetryTimeoutMS": 1000,
            "enable_duplication": false,
            "enable_mutation": false
        }
    // ...
}
```

Model



- **Import Model** (Button) - Import and Load state machine model according to `State Mapping` parameters defined on `configs/bt_config.json` . The supported formats are **.pcap, .pcapng** and **.json**
- **Save Model** (Button) - Save current state into both **.json** and **.dot** formats. The saved **.json** file can be later imported whereas **.dot** is used for state machine visualisation by any dot compatible viewer software.
- **Merge Modle** (Button) - Merge multiple state machine models into a single one. Similar to "Import Model" but you can select multiple files
- **Show All States** (Checkbox) - Whether to show all states or just few (previous and next states only) on the GUI state viewer window. This has no effect if the fuzzer is running without graphical interface (headless mode)
- **Enable Mapper** (Checkbox) - Enables State mapping during the fuzzing process. Whenever new transitions are not found during the fuzzing process, new paths will be created for the model.

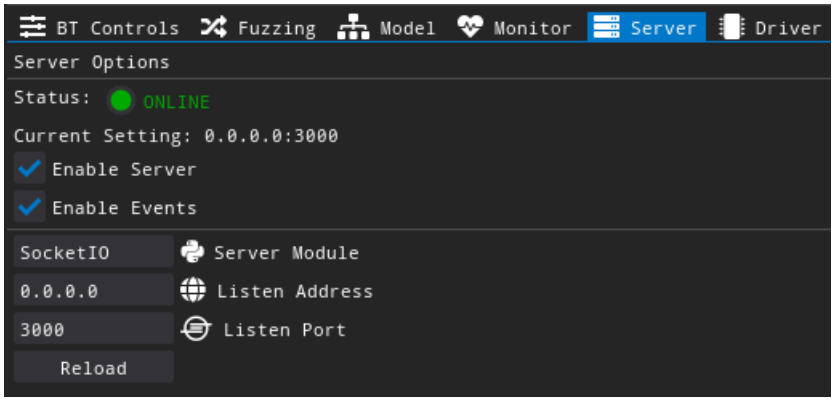
> **WARNING**
> **When using state mapper, always remember to save your model so you can use it later via "Import Model" button. Alternatively you can save the model with the same name of BT**

> **program so it loads automatically whenever the fuzzer starts. The default model path is `configs/models/bt/`**

The aforementioned model options are loaded from `configs/bt_config.json` with the following attributes:

```json
{
    "config": {
        // ...
        "StateMapper": {
            "Mapping": [
                {
                    "Filter": "btsdp",            // Filter rule
                    "LayerName": "SDP",           // Layer name indicated on state mach
                    "StateNameField": "btsdp.pdu" // Field name which contains the pack
                },
                // ...
            ],
            "SaveFolder": "configs/models/bt/",
            "ShowAllStates": true
        },
      // ...
    }
}
```

Server



- **Status** (LED) - Indicates if the server is running
- **Enable Server** (Checkbox) - Enables or disables the server. Toggling this has immediate effect.
- **Enable Events** (Checkbox) - Allow server to send event to clients. Disabling this requites that clients poll the server instead.
- **Server Module** (Text box) - Defines python script to use as the server on `modules/server` folder. Changing this requires pressing `reload` to take effect.
- **Listen Address** (Text box) - Defines the server listen address. Defaults to `0.0.0.0` to listen on all IPv4 addresses on local machine. Changing this requires pressing `reload` to take effect.

- **Listen Port** (Text box) - Defines the server listen port. Defaults to 3000. Changing this requires pressing `reload` to take effect.
- **Reload** (Button) - Reloads previous configures parameters except for "Enable Server" and "Enable Events" which take effect immediately.

The aforementioned server options are loaded from `configs/bt_config.json` on the following attributes:

```json
{
    "config": {
        // ...
        "ServerOptions": {
            "Enable": true,
            "EnableEvents": true,
            "ListenAddress": "0.0.0.0",
            "Logging": false,           // Allows server script to print log messages
            "Port": 3000,
            "Type": "SocketIO"
        },
      // ...
    }
}
```

⚙ Interface Drivers and Hardware Support

This sections describes in more detail the driver requirements (software or hardware) and the inner working of each driver, including their available APIs.

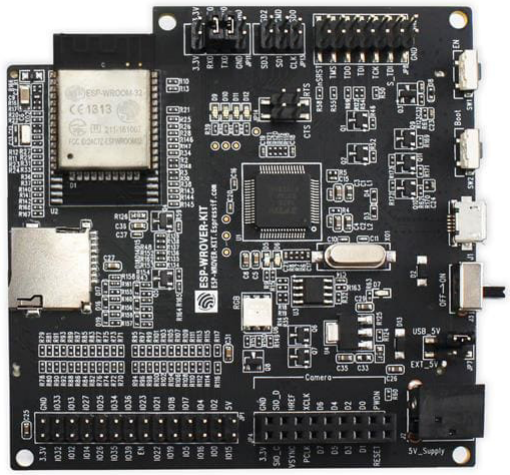
WDissector use drivers to communicate with the fuzzing interface. This interface can require a specific hardware or software or a combination of them.

Currently there are drivers for Bluetooth classic and LTE (OpenAirInterface). The current Drivers are listed on the table below.

Protocol Target	Interface Driver Name	Interface Communication	Hardware Support
Bluetooth Classic BLE Host	ESP32BTDriver	Serial Port over High-Speed USB	1. ESP-WROVER-KIT-VE
LTE / 4G (eNB)	OAICommunication	Shared Memory + Mutex	1. USRP B210
Wi-Fi 802.11	WifiRT8812AUDriver	Kernel Netlink	1. RT8812AU Compatible Wi-Fi Dongle. > Alpha AWUS036AC

ESP32BTDriver

Hardware



The recommended hardware is **ESP-WROVER-KIT-VE** . You can buy this board from several electronics suppliers and online stores such as:

- **Mouser**
- **Digikey**
- **LCSC**
- **Adafruit**
- **AliExpress**

Installing Driver

This driver relies on a custom firmware which must be flashed (installed) on a ESP32 Device. This procedure requires running a script which will automatically download the required tools to flash the custom firmware to ESP32 (via platformio framework). You can run the following on either `Windows 10` or `Ubuntu 18.04 / 20.04` .

Download the ESP32 firmware package as follows:

- Via **Web Browser** (Requires GitLab login): **https://gitlab.com/asset-sutd/software/wireless-deep-fuzzer/-/jobs/artifacts/wdissector/raw/release/esp32driver.zip?job=release**
- Or Via **bash command line**:

```sh
wget --header="PRIVATE-TOKEN: <gitlab token>" \
--content-disposition "https://gitlab.com/api/v4/projects/\
asset-sutd%2Fsoftware%2Fwireless-deep-fuzzer/jobs/artifacts/\
wdissector/raw/release/esp32driver.zip?job=release"
```

You can upload the firmware to ESP32 on either Windows or Linux. For windows you need to manually install python3 before executing the installation script.

Windows | Linux (Ubuntu)

```sh
sudo apt install unzip python3-dev
unzip esp32bt.zip # Extract esp32bt.zip if you haven't done so
cd release
python3 firmware.py /dev/ttyUSB0 # Please change your serial port to match your device
```

> **WARNING**
> You may need to press and hold the `boot` button during the firmware upload process if you get failures.

OAICommunication

Hardware

The LTE fuzzer, which uses OpenAirInterface for the LTE stack, requires use of an SDR. We recommend the use of USRP B210 which can be acquired directly from ETTUS . However, OAI also supports other SDRs which we didn't test yet.

Installing Driver

The custom OAI/Open5GS software which LTE fuzzer requires can be downloaded and built as follows:

```sh
# Go to wdissector root folder
cd 3rd-party
git clone https://gitlab.com/asset-sutd/software/oai_lte_network --depth=1
cd oai_lte_network/openair-ran
# Requirements will be installed to the system and build process will start
./build_all.sh
# Build Open5GS
cd ../../open5gs-core
./build.sh
```

☁ Web API

The fuzzer starts a SocketIO server and exposes access to its functions to a SocketIO client through websockets. The Table below lists the current API available for the client. **More to be added in future updates**.

Two types of server is available. You can choose the server type on the configuration file by changing the `ServerModule` index number:

```json
"ServerOptions": {
    "APINamespace": "/",            // Namespace (root path) of event. On REST, this pa
    "Enable": true,                 // Enable Server
    "EnableEvents": false,          // Enable SocketIO Asynchronous events (from server
    "ListenAddress": "127.0.0.1",   // Server address
    "Logging": false,               // Enable Server logging
    "Port": 3000,                   // Server port
    "ServerModule": 1,              // Server module index
    "ServerModulesList": [          // List of Server modules (from 0 to N)
        "SocketIOServer",           // modules/server/SocketIOServer.py (index 0)
        "RESTServer"                // modules/server/RESTServer.py     (index 1)
    ]
}
```

> **WARNING**
> SocketIO and REST Server events name are case sensitive. Requests from clients with wrong event names are ignored.

Request Events (Polling - SocketIOServer / RESTServer)

Event Name	Description	Input	Return	Example (SocketIO)
Summary	Returns summary of current fuzzing session, including statistics such as number of packets tarnsmitted and received	Nothing	**String**	scripts/server_test.py **-r** Summary

Event Name	Description	Input	Return	Example (SocketIO)
GraphDot	Returns current state machine in dot string format	Nothing	**String**	scripts/server_test.py **-r** GraphDot
GetModelConfig	Get fuzzer configuration	Nothing	**JSON**	scripts/server_test.py **-r** GetModelConfig
SetModelConfig	Set fuzzer configuration, returns parsing status	**JSON**	**Boolean**	scripts/server_test.py **-r** SetModelConfig -d '{"config":{...}}'
GetDefaultConfig	Get fuzzer default configuration	Nothing	**JSON**	scripts/server_test.py **-r** GetDefaultConfig
ResetConfig	Reset fuzzer configuration to default. Requires restart	Nothing	**JSON**	scripts/server_test.py **-r** ResetConfig
Shutdown	Shutdown fuzzer (terminate process)	Nothing	Nothing	scripts/server_test.py **-r** Shutdown
Start	Starts fuzzing the target defined in configuration file	Nothing	Nothing	scripts/server_test.py **-r** Start
Stop	Stops fuzzing	Nothing	Nothing	scripts/server_test.py **-r** Stop
Scan	Start BT scan (Enquiry) for a fixed time of 15 seconds. Subscribe to async event `Scan` to get targets results as soon as they are found during scanning	Nothing	Nothing	scripts/server_test.py **-r** Scan
GetScanResults	Get scan results of all targets found during latest scanning request. Request event `Scan` needs to be called first	Nothing	**JSON**	scripts/server_test.py **-r** GetScanResults
StartExploit	Start exploit by name	**String**	**Boolean**	scripts/server_test.py **-r** StartExploit **-d** knob
StopExploit	Stop exploit if it was previously running	Nothing	Nothing	scripts/server_test.py **-r** StartExploit **-d** knob

WARNING
REST Server implements `GET` and `POST` methods for all events (endpoints).

1. For `POST` requests, the argument must be sent in JSON type.
2. For `GET` the request data must be included in the URL the parameter `?args=`

Subscription Events (Asynchronous - SocketIOServer)

Event Name	Description	Return	Example (SocketIO)
Anomaly	Returns information of anomaly (crash/deadlock/anomaly)	**JSON**	scripts/server_test.py **-w** Anomaly
GraphUpdate	Returns string of current state in dot format	**String**	scripts/server_test.py **-w** GraphUpdate
Modules	Returns modules (exploits) status messages. The returned json object contains `"level"` to indicate priority (Green `"G"` , Yellow `"Y"` ,Red `"R"`) and `"msg"` with the status text.	**JSON**	scripts/server_test.py **-w** Modules
Scan	Returns scanned target information while scanning is in progress. The returned json object contains target `BDAddress` , `Name` , `RSSI` and `Class` .	**JSON**	scripts/server_test.py **-w** Scan

☢ Exploits

This section contains tables containing a summary of all exploits and their respective script file under `modules/exploits` folder.

Bluetooth Classic (BR/EDR)

Attacks

CVE ID	Attack Name	Affected Vendor(s)	Affected SoC(s) or Product(s)	Impact	Exploit Name
CVE-2021-28139	Feature Page Execution	Espressif Systems	ESP32 (SoC)	ACE / Deadlock	invalid_feature_page_execution.cpp
CVE-2021-28136	Duplicated IOCAP	Espressif Systems	ESP32 (SoC)	Crash (Reboot)	duplicated_iocap.cpp
CVE-2021-28135	Feature Res. Flooding	Espressif Systems	ESP32 (SoC)	Crash (Reboot)	feature_response_flooding.cpp
CVE-2021-28155	Feature Res. Flooding	Harman International	JBL TUNE500BT (Product)	Crash (Shutdown)	feature_response_flooding.cpp
CVE-2021-31609	LMP Auto Rate Overflow	Silabs	WT32i (SoC)	Crash (Reboot)	lmp_auto_rate_overflow.cpp
CVE-2021-34147	Invalid Timing Accuracy	Infineon Technologies (Former Cypress)	CYW20735B1 (SoC)	Crash (Reboot)	invalid_timing_accuracy.cpp
CVE-2021-34146	AU Rand. Flooding	Infineon Technologies (Former Cypress)	CYW20735B1 (SoC)	Crash (Reboot)	au_rand_flooding.cpp
CVE-2021-34145	LMP Invalid Max Slot Type	Infineon Technologies (Former Cypress)	CYW20735B1 (SoC)	Crash (Reboot)	invalid_max_slot.cpp

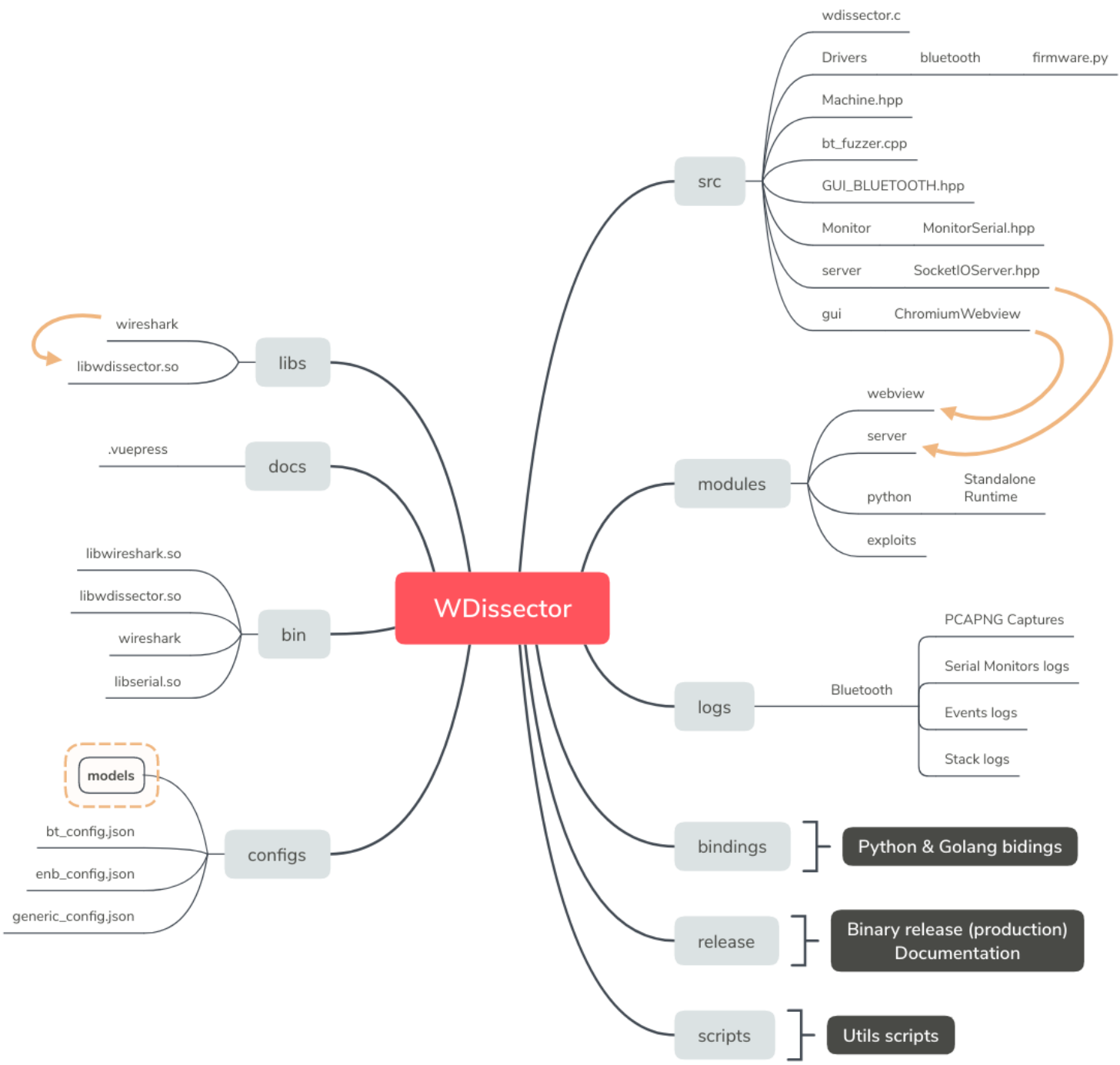
CVE ID	Attack Name	Affected Vendor(s)	Affected SoC(s) or Product(s)	Impact	Exploit Name
CVE-2021-34148	LMP Max Slot Overflow	Infineon Technologies (Former Cypress)	CYW20735B1 (SoC)	Crash (Reboot)	lmp_max_slot_overflow.cpp
CVE-2021-34149	AU Rand. Flooding	Texas Instruments	CC2564C (SoC)	Deadlock	au_rand_flooding.cpp
CVE-2021-31610	AU Rand. Flooding	Bluetrum	BT889X / AB5XX / AB5301A (SoCs)	Crash (Reboot)	au_rand_flooding.cpp
CVE-2021-34150	LMP Length Overflow over DM1	Bluetrum	AB5301A (SoC)	Deadlock (Paging disabled)	lmp_overflow_dm1.cpp
CVE-2021-34143	AU Rand. Flooding	Zhuhai Jieli Technology	AC6366C (SoC)	Deadlock	au_rand_flooding.cpp
CVE-2021-34144	Truncated SCO Link Request	Zhuhai Jieli Technology	AC6366C (SoC)	Deadlock	truncated_sco_link_request.cpp
CVE-2021-31612	LMP Auto Rate Overflow	Zhuhai Jieli Technology	AC6905X (SoC)	Deadlock	lmp_auto_rate_overflow.cpp
CVE-2021-31613	Truncated LMP_accepted	Zhuhai Jieli Technology	AC6905X / AC6925C (SoC)	Crash (Reboot)	truncated_lmp_accepted.cpp
CVE-2021-31611	Invalid Setup Complete	Zhuhai Jieli Technology	AC6905X / AC6925C (SoC)	Deadlock	invalid_setup_complete.cpp
CVE-2021-31787	Feature Res. Flooding	Actions Technology	ATS2815 / ATS2819 (SoC)	Crash (Shutdown)	feature_response_flooding.cpp
CVE-2021-31785	Repeated Host Connection	Actions Technology	ATS2815 / ATS2819 (SoC)	Deadlock	repeated_host_connection.cpp
CVE-2021-31786	Multiple Same Host Connection	Actions Technology	ATS2815 / ATS2819 (SoC)	Deadlock (Shutdown)	N.A (Specific BDAddress Configuration)
Pending	LMP Paging Scan Disable	Intel	Intel AX200 (SoC)	Deadlock (Paging disabled)	paging_scan_disable.cpp
Pending	Invalid Timing Accuracy	Intel	Intel AX200 (SoC)	Crash (FW Reboot)	invalid_timing_accuracy.cpp

CVE ID	Attack Name	Affected Vendor(s)	Affected SoC(s) or Product(s)	Impact	Exploit Name
CVE-2019-9506	KNOB	Intel	Intel AX200 (SoC)	Encryption Downgrade	knob.cpp
Pending	Invalid Timing Accuracy	Qualcomm	Snapdragon 845 / 855 / Others (SoCs)	Crash (FW Reboot)	invalid_timing_accuracy.cpp
Pending	LMP Length Overflow over 2-DH1	Qualcomm	CSR 8811 / CSR 8510 (SoCs)	Deadlock / Crash	lmp_overflow_2dh1.cpp
Pending	LMP Invalid Transport	Beken	BK3266	Deadlock (Paging disabled)	lmp_invalid_transport.cpp

Modules

WIP.

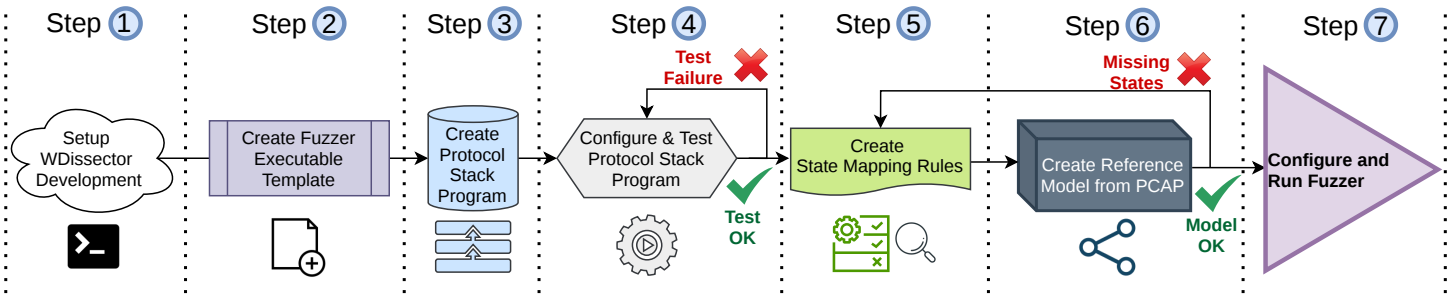
Architecture



Tutorials

* Adding a new protocol (BNEP Example)

This section guides your though adding fuzzing support to a new protocol into WDissector framework.



Keep in mind that you can easily add more protocols as long as **(1)** there is a compatible **Interface Drivers** that you can use to intercept communication for your protocol and **(2)** a **Protocol Stack Program** that can initiate communication using your protocol of choice.

For example, wireless protocol that relies on Bluetooth (L2CAP, SDP, RFCOMM) will use `ESP32BTDriver` , whereas protocols that are transported over ethernet (ICMP, IP, TCP, etc) can use either `WifiRT8812AUDriver` or `VirtualEthernet (Work-In-Progress)` . For your quick reference, below is listed what protocols are associated to which interface so you can get a feeling on what interface to use for an unlisted protocol:

- **ICMP, IP, TLS, DTLS, TCP, UDP, ARP, etc** - The `WifiRT8812AUDriver` can be used here since all such protocols are application layer. Although such Interface Driver is inted to be used for 802.11 data link and network layer protocol, nothing stops you to create specific fuzzing and mapping ruled to target application layer protocols. The stack used for Wi-Fi is normally **Hostapd** on Linux. Currently there is a work-in-progress to add a new Interface Driver `VirtualEthernet` which would allow the host to simply intercept a Virtual Ethernet interface on Linux and thus fuzz any Ethernet wired protocol down to the MAC layer (OSI Layer 2).
- **L2CAP, SDP, RFCOMM, BNEP, A2DP, GATT, AVRCP, etc** - The `ESP32BTDriver` is intended to be used with such Bluetooth related wireless protocols.

In this tutorial, we will show how to add fuzzing for **Bluetooth Personal Area Network (PAN)** profile using `ESP32BTDriver` Interface Driver. This profile uses the **Bluetooth Network Encapsulation Protocol** **(BNEP)** protocol and is used to establish internet connection to connected clients over Bluetooth. The internet connection is known to be slow due to *BT bitrate*, but our objective here is to just fuzz the **BNEP protocol**, so communication speed is not as important.

1) Prepare the development environment

In this step, you need to make sure that you have fully compiled from source wdissector at least once following the **Quick Start Guide**. In short, you can run the following commands on wdissector root folder:

```sh
./requirements.sh dev
./build.sh all
```

2) Creating a fuzzer executable for your protocol

We can use another fuzzer code as a template for us and starting point for our **BNEP protocol fuzzer executable**. Since `bt_fuzzer` uses `ESP32BTDriver` Interface, we can simply copy `src/bt_fuzzer.cpp` to a new `src/bt_pan_fuzzer.cpp` . We also do the same for it's configuration file as follows:

```sh
# Main fuzzer code \
cp src/bt_fuzzer.cpp src/bt_pan_fuzzer.cpp && \
# Main fuzzer config file \
cp configs/bt_config.json configs/bt_pan_config.json
```

Next, change the name of the json configuration file and the pcap capture file that our fuzzer is going to use. Open `src/bt_pan_fuzzer.cpp` and change the definition `CONFIG_FILE_PATH` to `"configs/bt_config.json"` and `CAPTURE_LMP_FILE` to `"logs/Bluetooth_PAN/capture_bluetooth.pcapng"` . You can use sed to do this change via terminal:

```sh
# Change config file name to bt_pan_config \
sed -i 's|bt_config.json|bt_pan_config.json|g' src/bt_pan_fuzzer.cpp && \
# Change log folder to Bluetooth_PAN \
sed -i 's|logs/Bluetooth/|logs/Bluetooth_PAN/|g' src/bt_pan_fuzzer.cpp
```

Then, we need to compile `bt_pan_fuzzer.cpp` as our new fuzzer. We will need to modify a bit its code later. For now, let's just make sure that we can compile our fuzzer within the framework by generating a fuzzer binary named `bt_pan_fuzzer` .

To do this, add the following at the end of the file `CMakeLists.txt` :

```
# Add this at the end of CMakeLists.txt
# Set PAN fuzzer source code files
set(BT_PAN_FUZZER_SRC src/bt_pan_fuzzer.cpp)
# Add PAN fuzzer executable
add_executable(bt_pan_fuzzer ${BT_PAN_FUZZER_SRC} libs/profiling.c)
target_link_libraries(bt_pan_fuzzer PRIVATE ${FUZZER_LIBS})
target_compile_options(bt_pan_fuzzer PRIVATE -w -O3)
target_compile_definitions(bt_pan_fuzzer PRIVATE -DFUZZ_BT)
target_precompile_headers(bt_pan_fuzzer PRIVATE src/PCHBT.hpp)
```

Finally, check if your fuzzer compiles successfully by running the following:

```
ninja -C build bt_pan_fuzzer
```

If everything goes well, you should see the following output on the terminal:

```
-- Generating done
-- Build files have been written to: /home/matheus/5g/wdissector/build
[1/6] cd /home/matheus/5g/wdissector/build/li...pl /home/matheus/5g/wdissector/libs/wireshark
We are not tagged.
version.h unchanged.
[6/6] Linking CXX executable ../bin/bt_pan_fuzzer
```

You can even run the generated `bt_pan_fuzzer` by running `sudo bin/bt_pan_fuzzer --help`. You should get a help message on the terminal. This is your **BNEP protocol fuzzer executable** and from now on, any changes to this fuzzer, means modifying `src/bt_pan_fuzzer.cpp` or its configuration file which will be discussed later.

3) Creating a BT Protocol Stack Program

Now that we have created a Fuzzer Executable, we need to create a Protocol Stack Program. The difference between both is that the Fuzzer Executable only modifies packets coming out of the `Interface Driver`, but it's the Protocol Stack Program that starts and handles the communication with the protocol intended to be fuzzed. For our BNEP protocol Fuzzer, we need to add a new program code to the folder `src/programs/bluetooth`.

We can rely on an existing BT program code example on the Bluekitchen stack called **pan_lwip_http_server.c**. Gladly, wdissector has such example file, which we can import into the framework as follows:

```sh
cp 3rd-party/btstack/example/pan_lwip_http_server.c src/programs/bluetooth/pan_lwip_htt
```

Similar to step **2)**, we need to tell our framework how to compile a new program in `CMakeLists.txt` . Run the following command to append the path of our new program to the `BT_PROGRAMS` list on `CMakeLists.txt` .

```sh
sed -i '/spp_counter.c/a ${PROJECT_SOURCE_DIR}/src/programs/bluetooth/pan_lwip_http_ser
```

This will add the following line in `CMakeLists.txt` .

```
# BT programs executable
set(BT_PROGRAMS
    ${PROJECT_SOURCE_DIR}/src/programs/bluetooth/spp_counter.c
${PROJECT_SOURCE_DIR}/src/programs/bluetooth/pan_lwip_http_server.c
    ${PROJECT_SOURCE_DIR}/src/programs/bluetooth/sdp_rfcomm_query.c
    ${PROJECT_SOURCE_DIR}/src/programs/bluetooth/sdp_general_query.c
    ${PROJECT_SOURCE_DIR}/src/programs/bluetooth/sdp_bnep_query.c
    ${PROJECT_SOURCE_DIR}/src/programs/bluetooth/a2dp_sink_demo.c
    ${PROJECT_SOURCE_DIR}/src/programs/bluetooth/a2dp_source_demo.c
    ${PROJECT_SOURCE_DIR}/src/programs/ble/gatt_browser.c)
foreach(BT_PROGRAM_FILE ${BT_PROGRAMS})
```

Next, we can compile our new PAN program as follows:

```sh
ninja -C build pan_lwip_http_server
```

If compilation is OK, you should see the program on `bin/pan_lwip_http_server` .

> **WARNING**
> This step may be optional for different combination of Interface Driver and protocol to be added. For example, you don't need to make WDissector framework compile the stack from scratch if you compiled it beforehand using a different method. You can instead pass directly to the fuzzer main code the path of the protocol stack binary so the fuzzer can start/restart it during the fuzzing session.

4) Configuring and Testing a Protocol Stack Program

Now, after we have a executable for our **Protocol Stack Program** from **Step 3)** that is located at `bin/pan_lwip_http_server` , we need to tell our fuzzer configuration file to start this program before fuzzing. To this end, we modify the parameters of our configuration file `configs/bt_pan_config.json` as follows:

- **Disable "PacketRetry" (false), "GlobalTimeout" and "LoopDetection"** - Our PAN program is a server and we don't want it to retry packets (*PacketRetry*) to the client. As for `"GlobalTimeout"` we set it to an extreme high value as we don't want to disconnect the client upon timeouts that may occur during our new protocol tests. Lastly, we also disable `LoopDetection` to not disconnect the client in case it send us multiple re-transmitted packets.

- **Disable "enable_duplication", "enable_mutation" and "enable_optimization" (false)** - We don't want our fuzzer to do fuzz or inject any packet yet. For now we are just testing the execution of our Protocol Stack Program through our Interface Driver, therefore we need to disable any fuzzing related parameter.
- **Add pan_lwip_http_server program to "DefaultPrograms"** - Here we add the path of our BT program to the list of default programs that the fuzzer can execute during startup. Add `"pan_lwip_http_server"` to the first item in the list and set parameter `"Program": 0` so the fuzzer starts our BT program. This step is crucial because the `Program` parameter selects the our program by its index in `"DefaultPrograms"` list, which is 0 for the first entry.

All of the above modifications can be quickly done by running the following commands:

```sh
# Disable "PacketRetry", "GlobalTimeout" and "LoopDetection" \
sed -i 's|"PacketRetry": true|"PacketRetry": false|g' configs/bt_pan_config.json && \
sed -i 's|"GlobalTimeout": 45|"GlobalTimeout": 9999|g' configs/bt_pan_config.json && \
sed -i 's|"LoopDetection": true|"LoopDetection": false|g' configs/bt_pan_config.json &&
# Disable "enable_duplication",.... \
sed -i 's|"enable_duplication": true|"enable_duplication": false|g' configs/bt_pan_cont
sed -i 's|"enable_mutation": true|"enable_mutation": false|g' configs/bt_pan_config.jsc
sed -i 's|"enable_optimization": true|"enable_optimization": false|g' configs/bt_pan_cc
# Add pan_lwip_http_server \
sed -i '/spp_counter/i "bin/pan_lwip_http_server",' configs/bt_pan_config.json && \
sed -i 's|"Program": 1|"Program": 0|g' configs/bt_pan_config.json
```

Next, we need to add a state machine model to our protocol. Since we don't have one yet, we copy an existent one just to test our protocol. The fuzzer won't use it for anything yet, but we need to provide a dummy one otherwise the fuzzer may hang during startup. The name of this dummy model must be the same as of our stack program (`pan_lwip_http_server`), but with *.json extension. You can create a dummy model as follows:

```sh
cp configs/models/bt/sdp_rfcomm_query.json configs/models/bt/pan_lwip_http_server.json
```

Finally, we can test our new Protocol Stack Program - Connect ESP32-WROVER-KIT-VE to your computer and start the fuzzer executable as follows:

```sh
sudo bin/bt_pan_fuzzer --no-gui --host-port=/dev/ttyUSB1 --autostart
```

If the ESP32 board is correctly connected at `/dev/ttyUSB1` . You should see the following output on the terminal:

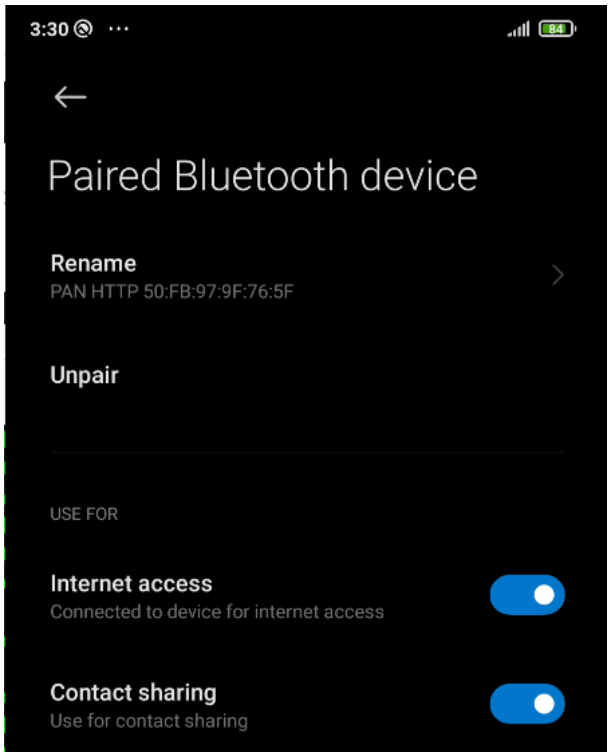
```
[BT Program] Starting program bin/pan_lwip_http_server -u /dev/pts/4 -a 98:DA:60:00:42:E3 --locap 3 --authreq 3 --boun
ding 1
Packet Log: logs/Bluetooth/hci_dump.pklg
H4 device: /dev/pts/4
lwIP version: 2.2.0d
SDP service record size: 218
Local version information:
- HCI Version    0x0008
- HCI Revision   0x030e
- LMP Version    0x0008
- LMP Subversion 0x030e
- Manufacturer 0x0060
Local name:
[Baseband] TX --> FHS
[L2CAP] TX --> Connection oriented channel
BTstack up and running at 26:EE:CE:47:3B:D3
```

Such indicates that the new BT PAN program is running.

We can now test if communication is working by connecting from our smartphone to the PAN server with the BDAddress indicated in the terminal:

1. Disable Wi-Fi on your Smartphone and go to the Bluetooth scanning screen. You should see the device "**PAN HTTP XX:XX:XX:XX:XX:XX**" listed as below:

2. Go to the BT device list, access **PAN HTTP** device details or options and enable "Internet Access":



3. At this point, if internet access was enabled, it means that your phone is already communicating with internet via PAN and the protocol program was successfully created and configured. To confirm this you can also take a look on the fuzzer terminal. It should show many messages excanges, indicating now **TCP/IP/DNS**:

```
[ICMPv6] Router Solicitation from a4:50:46:59:0c:90
[ARP] Who has 192.168.7.1? Tell 192.168.7.2
[ARP] 192.168.7.1 is at 50:fb:97:9f:76:5f
[TCP] 49510 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1 TSval=66394223 TSecr=0 WS=512
[LMP] RX <-- LMP_set_AFH
[TCP] [TCP Out-Of-Order] 49510 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1 TSval=66394328 TSecr=0 WS=512
[TCP] [TCP Out-Of-Order] 49510 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1 TSval=66394528 TSecr=0 WS=512
[DNS] Standard query 0xcf68 A e8.whatsapp.net
[DNS] Standard query 0xf436 AAAA e8.whatsapp.net
[LMP] RX <-- LMP_sniff_req
[LMP] TX --> LMP_not_accepted
[TCP] [TCP Out-Of-Order] 49510 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1 TSval=66394928 TSecr=0 WS=512
[TCP] 39264 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1 TSval=66395685 TSecr=0 WS=512
[TCP] [TCP Out-Of-Order] 49510 → 443 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1 TSval=66395776 TSecr=0 WS=512
[TCP] [TCP Out-Of-Order] 39264 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1 TSval=66395792 TSecr=0 WS=512
[TCP] [TCP Out-Of-Order] 39264 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1 TSval=66395992 TSecr=0 WS=512
[ARP] Who has 192.168.7.1? Tell 192.168.7.2
[ARP] 192.168.7.1 is at 50:fb:97:9f:76:5f
[TCP] [TCP Out-Of-Order] 39264 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM=1 TSval=66396392 TSecr=0 WS=512
```

4. You can now close the fuzzer with Ctrl-C and open the capture file that was saved on `logs/Bluetooth_PAN/capture_bluetooth.pcapng` . The capture file should appear many BT Exchanges alongside IP packets such as shown below:

No.	Source	Destination	Protocol	Info
213			LMP	RX <-- LMP_features_res_ext
214			L2CAP	Sent Connection Response - Success (SCID: 0x0055, DCID: 0x0044)
215			L2CAP	Sent Configure Request (DCID: 0x0055)
216			L2CAP	Rcvd Configure Request (DCID: 0x0044)
217			L2CAP	Rcvd Configure Response - Success (SCID: 0x0044)
218			LMP	RX <-- LMP_preferred_rate
219			BNEP	Rcvd Control - Setup Connection Request - dst: <PAN NAP>, src: <PAN PANU>
220			BNEP	Sent Control - Setup Connection Response - Operation Successful
221	::	ff02::1:f…	ICMPv6	Neighbor Solicitation for fe80::a650:46ff:fe59:c90
222	::	ff02::16	ICMPv6	Multicast Listener Report Message v2
223	::	ff02::1:f…	ICMPv6	Neighbor Solicitation for fe80::619a:3410:f25:43cd
224	0.0.0.0	255.255.2…	DHCP	DHCP Discover - Transaction ID 0x3ffde59b
225	192.168.7.1	255.255.2…	DHCP	DHCP Offer - Transaction ID 0x3ffde59b
226			ACL	TX --> Fragment
227	::	ff02::16	ICMPv6	Multicast Listener Report Message v2
228	0.0.0.0	255.255.2…	DHCP	DHCP Request - Transaction ID 0x3ffde59b
229	192.168.7.1	255.255.2…	DHCP	DHCP ACK - Transaction ID 0x3ffde59b

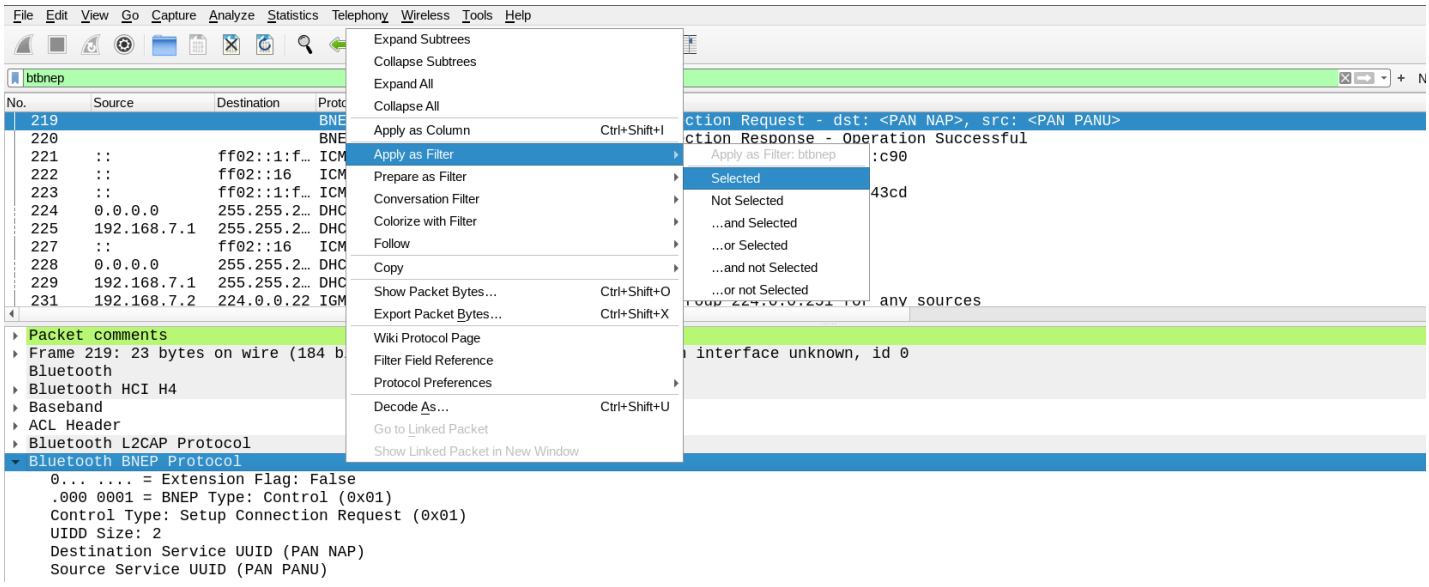
5) Analysing Protocol Capture and Creating State Mapping Rules

Now that we have a **(1)** working **Protocol Stack Program** and a **(2) Fuzzing Executable** which can communicate with a target via a **Interface Driver**, we can proceed to create a simple reference protocol model for our fuzzing.

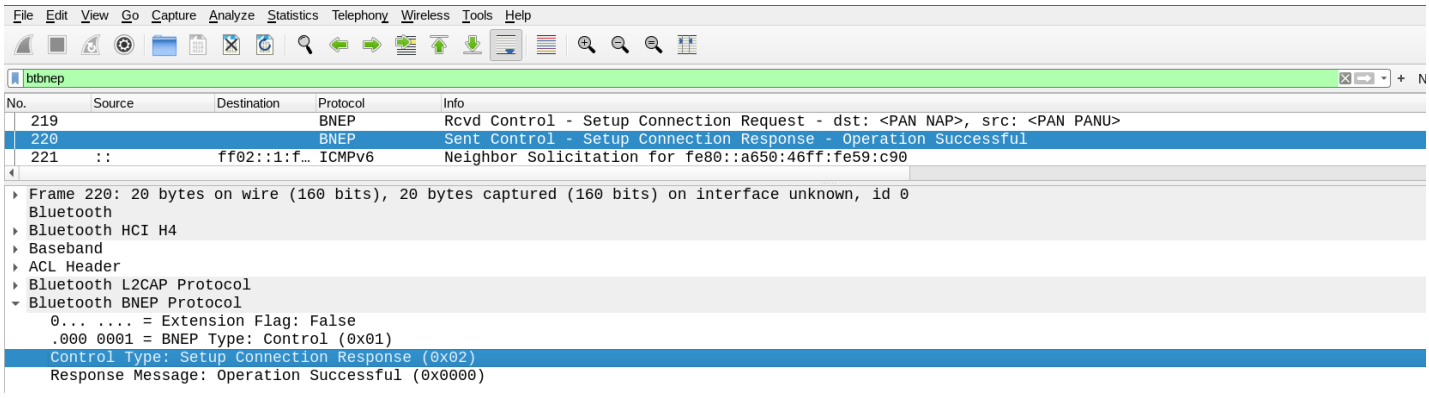
To this end, note that we will use the capture file obtained from **Step 4)**. If you didn't complete last step but would like to try the reference model generation, you can use the example file located at `examples/wdmapper/capture_bt_pan.pcapng` .

1. First, we need to create some rules (*Mapping Rules*) on how to identify our protocol in the packet exchanges. To this end, open the capture file `logs/Bluetooth_PAN/capture_bluetooth.pcapng` from **Step 4)** and apply the filter **btbnep** in the Wireshark filter text-box to facilitate our analysis of the BNEP protocol. You can alternatively right click the protocol layer on the Wireshark tree view and select "Apply as Filter". This will only show packets that contain **BNEP header**, which is used in the

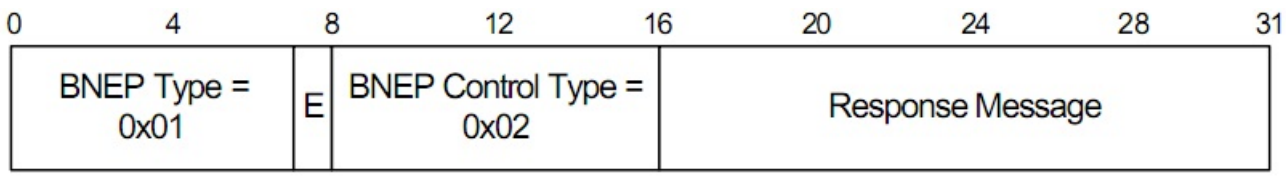
PAN profile.

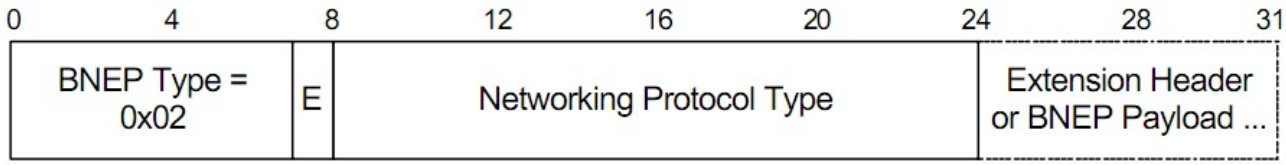


2. Next, after filtering only BNEP packets, it's important to located which field in the BNEP header corresponds to its packet type. Because BNEP protocol adds just a single header and has few fields, it's easy to locate a field called **Control Type** in Wireshark. We can see that our second packet is listed as *Setup Connection Response* in the **Control Type field**.



This Control type field is what we want since it identifies the type of BNEP packet and thus can be used for state mapping. However, if we do a quick search on BNEP packet header structure, we see that the **Control Type** field only exists if the ***BNEP Type* field is (0x01)**. Otherwise, if ***BNEP Type is 0x02***, the *Control Type* field is not present.

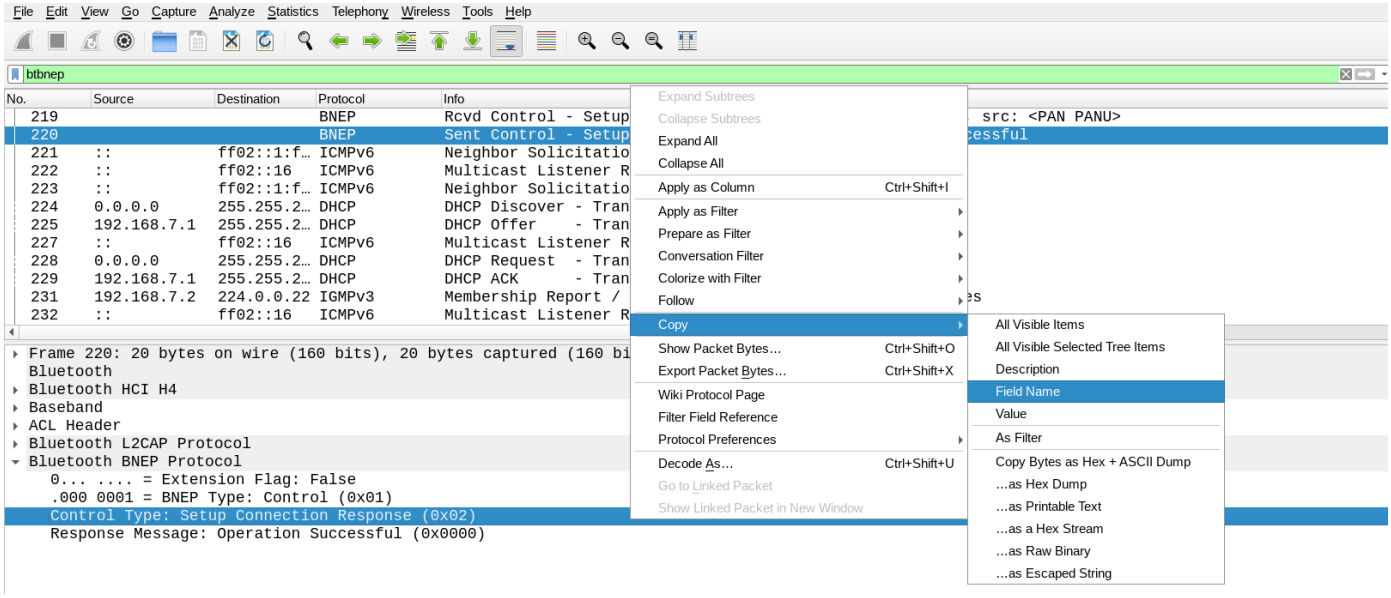




Therefore, we need to somewhat use both fields (*BNEP Type* and *Control Type* fields for our state mapping). The good thing is that it's easy to do so in our state mapping configuration file.

3. Now that we have some clue that we need to use both *BNEP Type* and *Control Type* fields, we fist need to know what is their reference name in wireshark so we can add them to our fuzzing

configuration file (*bt_pan_config.json*) for state mapping. We can get the reference name of such field by right clicking them on Wireshark tree view and selecting "Copy->Field Name" as illustrated below:



Doing this with both *BNEP Type* and *Control Type* fields, we can get the reference names **"btbnep.bnep_type"** for *BNEP Type* and "**btbnep.control_type**" for *Control Type*.

4. Now that we know the reference names for our type fields (**"btbnep.bnep_type"** and "**btbnep.control_type**"), we need to add them to our fuzzer configuration file `bt_pan_config.json` . Next, open `bt_pan_config.json` with a text editor and write the following in the **"Mapping"** property:

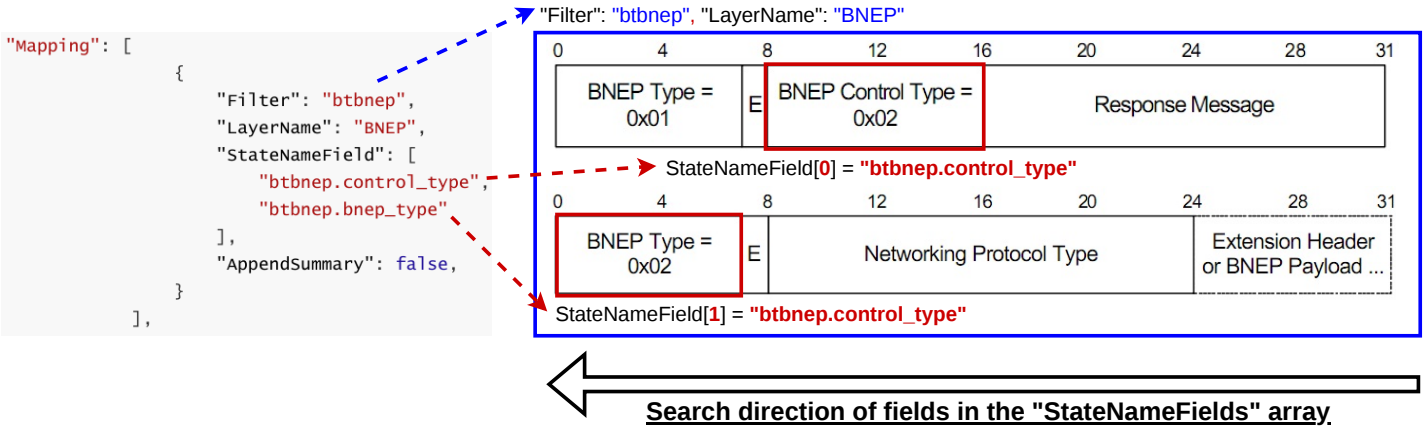
```json
"Mapping": [
                {
                    "Filter": "btbnep",
                    "LayerName": "BNEP",
                    "StateNameField": [
                        "btbnep.control_type",
                        "btbnep.bnep_type"
                    ],
                    "AppendSummary": false,
                }
            ],
```

Note that there is some parameters that is worth mentioning here:

- **Filter** - This parameters indicates to the State mapper what is the protocol, within a packet, that we want to filter. If a packet contains the protocol indicated by this filter, then the mapper checks if any ***"StateNameField"*** exists in the packet and *label* this packet layer with ***"LayerName"*** parameter. In this example, we use the same filter we have used in the start of this step to filter BNEP packets in Wireshark (i.e., **"btbnep"**).

- **LayerName** - Arbitrary name for the protocol matching the ***"Filter"*** parameter. In this example we use **"BNEP"** because this is the protocol we are interested in mapping. In reality you can use any name for this parameter.
- **StateNameField** - This is the most important parameter and is the one that tells the mapper what fields to use to classify the packet in a state. In this step, we have identified ***"btbnep.control_type"*** and ***"btbnep.bnep_type"*** as the *StateNameFields*.
- **AppendSummary** - Not used here. It's useful when refining the state machine, in case the packet type alone does not return meaningful strings to identify a state. In such case we can add the *Info* Column of Wireshark (packet summary) as the label for the packet. In most cases you can leave this option as `false` .

A summary of how all those parameters relate to our chosen protocol is illustrated below:



Note that the order of the fields inside the *StateNameField* array is important. Since the State Mapper stops the field search on the first match, the *StateNameField* should contain fields that are positioned from right (end) to left (start) in the protocol bytes. Simply put, you can look on Wireshark what field is the last and place it on the first position of the *StateNameField* array. This restriction may be a bit hard to grasp at first glance, but this field ordering issue can be better understood when visualising the created state machine model.

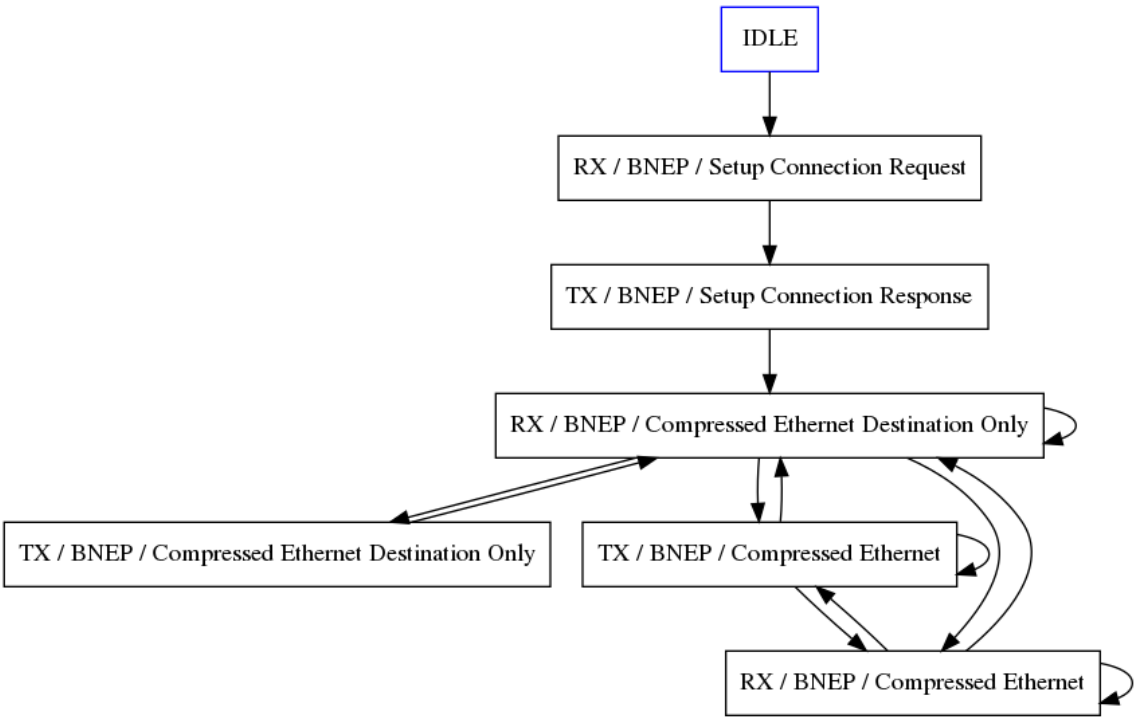
6) Creating a Reference Model File by Running State Mapper

1. Finally, we can generate and visualise the model by using the State Mapper (*wdmapper*). *wdmapper* receives few arguments: It requires as input the pcapng capture file (**-i**); the fuzzer configuration file (**-c**) and output the state model (**-o**). More details on each argument can be found by running help: `bin/wdmapper --help` . In short, we can run the following command to generate the protocol state machine model:

```sh
# Generate image
bin/wdmapper -i logs/Bluetooth_PAN/capture_bluetooth.pcapng -c configs/bt_pan_config.js
# Generate json (required)
bin/wdmapper -i logs/Bluetooth_PAN/capture_bluetooth.pcapng -c configs/bt_pan_config.js
```

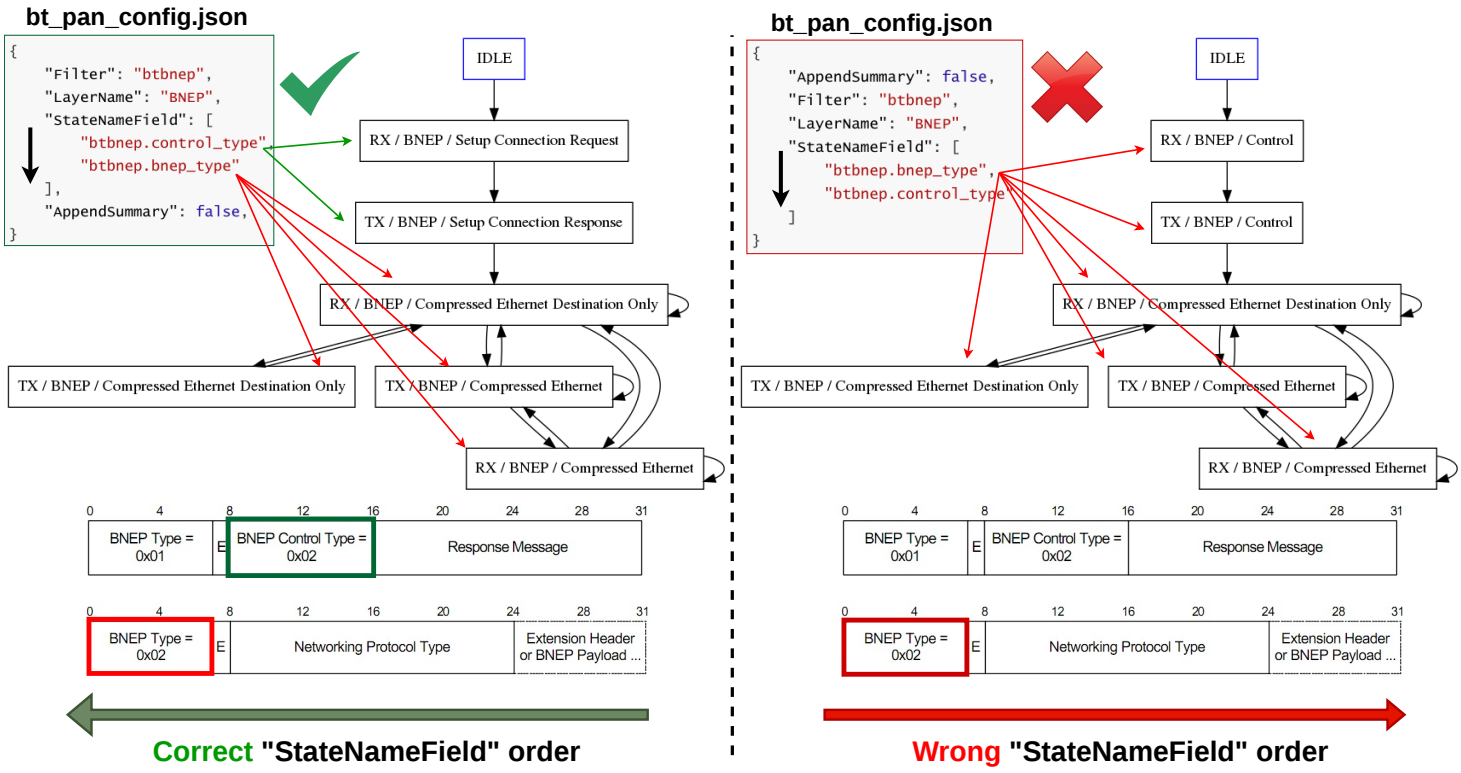
The first command is to generate a figure in .png format, while the second **(.json)** generates the actual model which is used by the fuzzer.

If the mapper is successful, the figure of the state machine `model_map.png` is created at your current working directory, which should be *wdissector* root folder:



By visually inspecting the generated state machine model and comparing with our original capture file on Wireshark from **Step 5)**, we can confirm that the above model looks OK as the *BNEP* protocol header is simple. Nevertheless, it's recommended to always review the model graph to ensure that the fuzzer will not miss any relevant states during fuzzing. If there is any state missing, it's adivisable to go back to **Step 5)** to fix the mapping rules.

A common mistake that could happen at this step is to write the *"StateNameField"* in the wrong order. An example of this issue is shown below:



In the Figure above, it's possible to see how the order of field references in *"StateNameField"* affects the reference model generation. On the left side, the mapper correctly matches the *Control Type* before *BNEP Type* and can label the first two states as *Setup Connection Request* and *Setup Connection Response*. On the other hand, the right side makes the mapper label the first two state as *Control* since it matches all packets with the *BNEP Type*, therefore, missing information on what specific control packet it's being exchanged during BNEP communication.

2. If the model looks fine as mentioned before, copy the **.json** model to the correct folder so our BNEP fuzzer can load our new model during startup. Previously in **Step 4)**, we had copied a dummy *pan_lwip_http_server.json* model for our protocol tests, but now we are inserting the correct model by copying our generated model `model_map.json` to `configs/models/bt/pan_lwip_http_server.json` :

```sh
cp model_map.json configs/models/bt/pan_lwip_http_server.json
```

> **WARNING**
> For this example we have only used one capture file to create the model. For other more complex protocols, it's advisable to generate more sample captures from target devices to ensure robustness of the state machine model.

7) Configuring and Running our new Fuzzer

Now, we are almost ready to start fuzzing our new protocol target, but there is still some fuzzing-related changes to be made on our configuration file `configs/bt_pan_config.json` :

- Allow fuzzing only BNEP packets (using btbnep filter).
- Adjust Packet Layer Fuzzing Offset ("**PacketLayerOffset**") in *configs/bt_pan_config.json* and *src/bt_pan_fuzzer*.
- Enable fuzzing modes ("**enable_***") in *configs/bt_pan_config.json*.

```sh
# Adjust Packet Layer Fuzzing Offset "PacketLayerOffset" \
sed -i 's|"PacketLayerOffset": 1|"PacketLayerOffset": 4|g' configs/bt_pan_config.json &
sed -i 's|packet_navigate_cpp(2, 1,|packet_navigate_cpp(5, 0,|g' src/bt_pan_fuzzer.cpp
# Enable fuzzing modes \
sed -i 's|"enable_duplication": false|"enable_duplication": true|g' configs/bt_pan_cont
sed -i 's|"enable_mutation": false|"enable_mutation": true|g' configs/bt_pan_config.jsc
sed -i 's|"enable_optimization": false|"enable_optimization": true|g' configs/bt_pan_cc
```

Allow fuzzing only BNEP packets

The fuzzer needs to exclude any packet that does not contain BNEP protocol. To this end, we simply need to write the following *Exclusion rule* to the **"Excludes"** property in `configs/bt_pan_config.json` :

```json
"Excludes": [{
    "ApplyTo": "A",
    "Description": "Ignore non BNEP packets",
    "Filter": "!btbnep"
}],
```

Note that the exclusion parameter **"Filter"** uses the same sintax of wireshark filter (*epan dfilter*). Therefore, if you are unsure what to write in this field, simply add a exclamation **(!)** in the front of the filter name that only shows packet of your target protocol. In our case, we are using **btbnep** for BNEP protocol, so the exclusion filter becomes the negation: **!btbnep**. In other words: *Exclude from fuzzing all packets that do not contain BNEP protocol*.

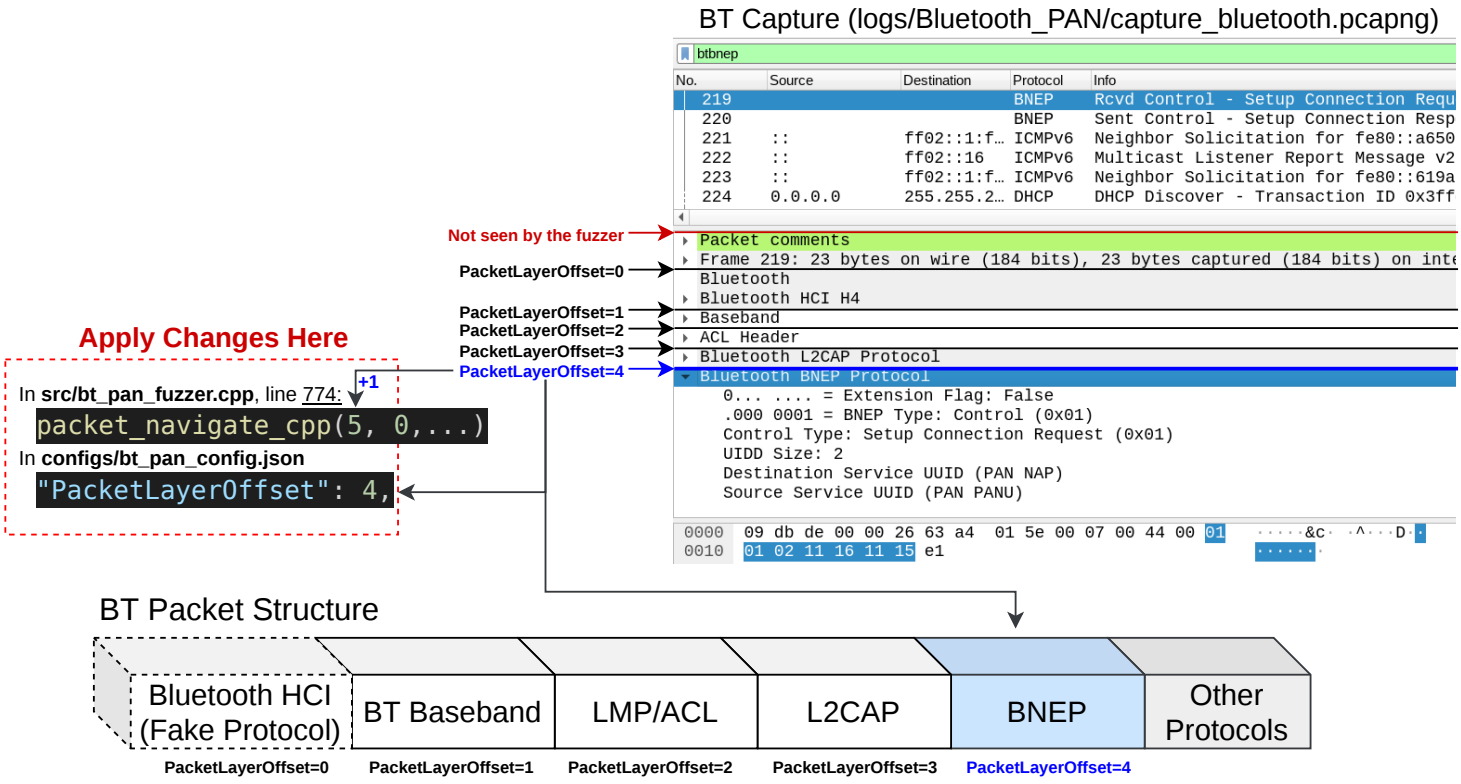
Adjusting the Packet Layer Fuzzing Offset (*PacketLayerOffset*)

We need tell the fuzzer *where* is the initial protocol layer to start fuzzing a packet. Because BNEP Protocol is encapsulated by other protocol such as LMP, L2CAP, we need to fuzz bytes that BNEP is located further in the transmitted packet bytes. This is because our *Interface Driver* intercepts the full over-the-air packet and not just partial protocols PDUs.

To understand such *offset*, we need to go back to our saved protocol capture that we have analysed in Step 5) and open the capture file `logs/Bluetooth_PAN/capture_bluetooth.pcapng` . As shown in the Figure below, each protocol layer in the *BT Packet Structure* corresponds to a ***"PacketLayerOffset"***

index. This is highlighted in the Wireshark tree view. Note that the layer *Frame* in the Wireshark tree view is ignored by the fuzzer so it doesn't have a corresponding *PacketLayerOffset*.

- Since BNEP is our target protocol, we write **"PacketLayerOffset": 4** in the configuration file *configs/bt_pan_config.json* and modify the source code of our fuzzer executable *src/bt_pan_fuzzer.cpp* on *line 774*: Changing the first and second arguments in the function *packet_navigate_cpp* to **(5, 0)**. The first argument is **5** because it must follow the formula *PacketLayerOffset+1*. On future updates of the frameworks, these manual modifications of the fuzzer source code are expected to be removed. For now we are manually doing the changes for a new protocol.



Enabling Fuzzing modes

On **Step 4** we have disabled the parameters `enable_*` to test our protocol communication, but now we intend to finally start the fuzzer executable with the actual packet fuzzing enabled. As a reminder, the parameters that we are enabling are:

- **enable_mutation** - Enable packet fields mutation;
- **enable_duplication** - Enable sending packets out of order (duplication);
- **enable_optimization** - Enable fuzzing optimization. This is intended to improve the packet mutation process. This parameter only works if `enable_mutation` is `true` .

Running the fuzzer

Compile our *fuzzer executable* to make sure our changes in `src/bt_pan_fuzzer.cpp` were commited.

```sh
ninja -C build bt_pan_fuzzer
```

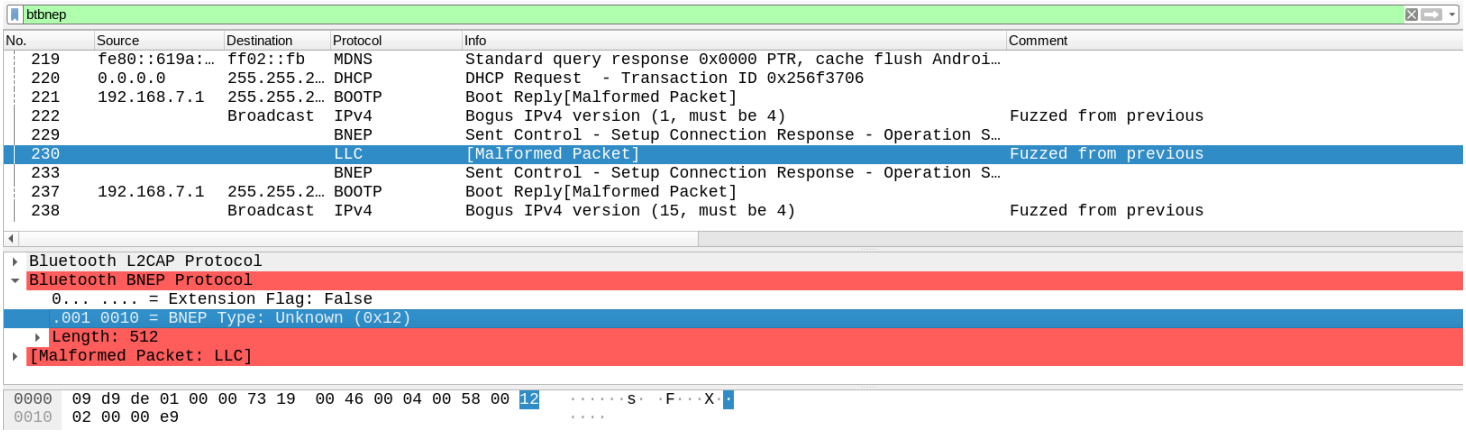
Then finally run our fuzzer as shown below:

```sh
sudo bin/bt_pan_fuzzer --no-gui --host-port=/dev/ttyUSB1 --autostart
```

You will need to connect to the BNEP fuzzer via your phone. After repeating the testing methods discussed in **Step 4**, You should be able to see on the terminal purple messages indicating that a transmitted packet has been fuzzed such as shown below:

```
[L2CAP] TX --> Disconnection Response (SCID: 0x004f, DCID: 0x0043, PSM: 0x000f, Service: BNEP)
[L2CAP] RX <-- Connection Request (BNEP, SCID: 0x0050)
[L2CAP] TX --> Connection Response - Success (SCID: 0x0050, DCID: 0x0044)
[L2CAP] TX --> Configure Request (DCID: 0x0050)
[L2CAP] RX <-- Configure Request (DCID: 0x0044)
[L2CAP] RX <-- Configure Response - Success (SCID: 0x0044)
[BNEP] RX <-- Control - Setup Connection Request - dst: <PAN NAP>, src: <PAN PANU>
BNEP connection open succeeded to A4:50:46:59:0C:90 source UUID 0x1115 dest UUID: 0x1116, max frame size 1676
Please open 'http://192.168.7.1' in your web browser:
Fuzzed 1 fields : TX --> Control - Setup Connection Response - Operation Successful
[BNEP] TX --> Control - Setup Connection Response - Operation Successful
[BNEP] RX <-- Control - Command Not Understood - Unknown(Unknown type)
BNEP channel closed
[L2CAP] TX --> Disconnection Request (SCID: 0x0044, DCID: 0x0050, PSM: 0x000f, Service: BNEP)
[L2CAP] RX <-- Disconnection Response (SCID: 0x0044, DCID: 0x0050, PSM: 0x000f, Service: BNEP)
[LMP] RX <-- LMP_preferred_rate
[LMP] RX <-- LMP_detach
```

Additionally, you can also take a look on the saved captured file at `logs/Bluetooth_PAN/capture_bluetooth.pcapng` . During experimentation, by enabling and disabling the "Internet Access" checkbox on the Bluetooth device screen, it's possible to repeat multiple BNEP re-connections. After such connection retries, it's possible to get the initial BNEP configuration packets to be mutated as shown in the Figure below. In this case, the BNEP Type fields has been mutated to *0x12*, which is an invalid value. On the phone, it's noticeable that the BNEP "Internet Access" checkbox is greyed-out, indicating some connection problem. Certainly this is exactly the sort of behaviour that we expect from the client (Android) during fuzzing!



> **WARNING**
> As a final note, if you are not getting much packets to be fuzzed during your BNEP reconnections, try to disable optimization on the configuration file (enable_optimization: false) and increase the value of parameters "DefaultMutationFieldProbability" and

> "DefaultDuplicationProbability" to 0.5 for example. This forces the fuzzer to use the same random chance to fuzz all BNEP packets. With 0.5 (50%) of mutation chance, certainly you can get the initial BNEP packets to be fuzzed within 1-2 reconnections.

Discussion

This concludes this tutorial on creating a new protocol from scratch using wdissector. There are some other aspects to the framework which was not discussed here such as *device monitoring*, *protocol stack program auto-restart* and *Interface Driver API and usage*. Hopefully we will add more tutorials as the framework evolves.

* Adding new server API events

Adding new events to the server requires changing the fuzzer code to register new function to corresponding events. The following code example below shows how to use `src/PythonServer.hpp` C++ library to easily add callbacks to events and get parameters sent from clients. Note the function `RegisterEventCallback` requires a string as the event name and a function pointer as callback. The example uses lambda expression to allow us to declare the callback as the argument itself.

> **WARNING**
> While C++ SocketIO event callbacks are busy, the main Python interpreter is blocked due to the Global Interpreter Lock (GIL). As such, do not place any time consuming code inside such C++ events. Otherwise, background Python threads will not work as expected or freeze while C++ code is being executed.

```
#include <iostream>
#include <string>
#include "server/SocketIOServer.hpp"

using namespace std;

SockerIOServer Server; // Server Instance

// Initialize Python Runtime
if (PythonCore.init())
{
    if (Server.init("0.0.0.0", 3000)) // Server starts here on port 3000
    {
        cout << Server.TAG << "Server at " << Server.listen_address << ":" << Server.po

        // Register callback to client connection or disconnection
        Server.OnConnectionChange([](bool connection) {
            if (connection)
                cout << Server.TAG, "Remote Client Connected" << endl;
```

```
            else
                cout << Server.TAG, "Remote Client Disconnected" << endl;
        });
        // Example 1: No arguments (simply do not use "args")
        Server.RegisterEventCallback("Example1", [&](py::list args) {
            // Perform action to event "Example1" requested by some client
        });
        // Example 2: Read arguments (convert first argument to string from "args")
        Server.RegisterEventCallback("Example2", [&](py::list args) {
            if (args.size() > 0)
            {   // Convert first argument from arguments array "args" to string
                string config_string = args[0].cast<string>();
                // Do something ...
            }
        });
        // Example 3: Return something back to the client
        Server.RegisterEventCallback("Example3", [&](py::list args) -> string {
            // Do something ...
            return "Hello back"
        });
    }
    else
    {
        GL1R(Server.TAG, "Server failed to initialize");
    }
}
```

© Licenses

WIP.